

PROGRAMACIÓN III · 3.4.077

Trabajo Práctico Obligatorio

Segundo cuatrimestre 2026 · Comisión Miércoles Mañana · Docente: Manuel Adrián Cáceres

Cómo funciona esta materia

El TPO no es una entrega de fin de cuatrimestre: es la columna vertebral de la cursada. Arranca en la primera clase y cada clase le suma un algoritmo. Las actividades de clase SON los hitos del trabajo.

Aprobar el TPO es requisito para rendir el Segundo Parcial, que consiste en escribir código frente al docente.

1. Qué hay que hacer

Desarrollar un sistema completo con Spring Boot y Neo4j sobre un dominio propio, implementando los algoritmos vistos en clase. El punto de partida es el repositorio scaffold de la materia, que provee el modelo, la conexión a la base y todos los endpoints ya cableados, pero ningún algoritmo implementado.

Scaffold: github.com/Mancaceresuade/tpo-scaffold-2026

1.1 El dominio es libre, pero tiene que soportar los algoritmos

Cada grupo elige su dominio en el hito 3. La única restricción es que el grafo resultante permita aplicar los algoritmos con sentido. Antes de elegir, respondan estas tres preguntas:

1. ¿Qué es una arista y qué número tiene encima? Si no pueden completar la frase «la conexión entre X e Y cuesta N ____», el dominio no sirve: sin peso numérico no hay Dijkstra, ni Prim, ni Kruskal.
2. ¿Tiene sentido preguntar cuál es el camino más barato de A a B pasando por nodos intermedios?
3. ¿Hay algo que seleccionar bajo una restricción de capacidad?

Advertencia sobre dominios bipartitos

Los dominios del tipo «jugadores y equipos», «alumnos y materias» o «productos y categorías» modelan relaciones de PERTENENCIA, no de conexión. El grafo queda bipartito y sin peso: sobre eso no se pueden implementar Dijkstra, Prim ni Kruskal, que son 3 de los 10 puntos.

Regla práctica: si el grafo se dibuja con dos columnas de nodos y flechas de izquierda a derecha, elijan otro dominio.

Dominios que funcionan bien

Dominio	Nodo	Costo de la arista	Item
Red logística	depósito, punto de entrega	km o minutos	paquete (peso, valor)
Transporte urbano	estación	tiempo del tramo	pasajero (equipaje, prioridad)
Red eléctrica	subestación	costo de tendido	carga a abastecer
Red de datos	servidor, router	latencia del enlace	job (CPU, ganancia)
Turismo	ciudad	precio del pasaje	actividad (horas, puntaje)

2. Cómo se evalúa la cursada

Hay tres instancias con nota. La nota de cursada es el promedio simple de las tres, y las tres deben superar los 4 puntos.

Instancia	Fecha	Formato	Qué mide
Primer Parcial	16/09	Escrito, individual	Conceptos: complejidad, recurrencias, elección de técnica
TPO	28/10	Grupal, con hitos	Construcción del sistema y de los algoritmos
Segundo Parcial	11/11 y 18/11	Codificación en vivo, individual	Que sabe escribir los algoritmos, no solo describirlos

El encadenamiento

Cumplir los hitos habilita a aprobar el TPO. · Aprobar el TPO habilita a rendir el Segundo Parcial.

El Segundo Parcial cumple además el requisito de defensa individual del Art. 12° de la normativa académica, por eso no hay un coloquio separado.

3. Los hitos

Cada clase de dictado tiene un hito. No llevan nota individual: se registran como cumplido o no cumplido, y se entregan por Teams antes de la clase siguiente.

Para aprobar el TPO hay que cumplir al menos **9 de los 12 hitos**, y los hitos 3, 4, 5 y 12 son obligatorios sin excepción.

#	Fecha	Hito	Qué hay que entregar
1	05/08	Entorno corriendo	Clonar el scaffold, levantar Neo4j y que GET /api/grafos/resumen responda con 8 vértices y 12 aristas. Captura de pantalla en Teams.
2	12/08	Ordenamiento propio	QuickSort y MergeSort implementados sobre los items del grafo semilla, sin usar Collections.sort. Un test por cada uno.
3	19/08	Grupo y dominio	Grupo conformado (3 o 4). Una página en el README: dominio elegido, qué es un nodo, qué es una arista, qué representa el costo y en qué unidad. Diagrama con al menos 8 nodos y sus pesos.
4	26/08	Dominio cargado y recorridos	El dominio propio cargado en Neo4j reemplazando la semilla. BFS y DFS funcionando sobre él.
5	02/09	Caminos mínimos y MST	Dijkstra con reconstrucción de camino, Prim y Kruskal con Union-Find. Los tres verificados contra un cálculo hecho a mano sobre el grafo del grupo.
6	09/09	Greedy contra PD	Mochila 0/1 con tabla dp y recuperación del camino. Documentar un juego de datos donde el greedy del hito 3 NO dé el óptimo.
7	23/09	Todos contra todos	Floyd-Warshall sobre el grafo del grupo, con detección de ciclos negativos. Comparar los nodos expandidos contra correr Dijkstra V veces.
8	30/09	Exploración con poda	Backtracking de rutas simples con restricción de costo y de saltos. Reportar nodos explorados con poda y sin poda.
9	07/10	Tests y documentación	Un test por cada algoritmo implementado hasta acá. docs/ENDPOINTS.md completo con complejidad justificada de cada endpoint.
10	14/10	Informe de complejidades	Tabla con los algoritmos del trabajo, su recurrencia cuando corresponda, su complejidad temporal y espacial, y la estructura de datos que la sostiene.
11	21/10	Optimización con cota	Branch & Bound de reparto. Explicitar cuál es la cota y por qué es optimista. Reportar nodos podados.

#	Fecha	Hito	Qué hay que entregar
12	28/10	Entrega final	Repositorio con el tag entrega-final. Cumple los cinco requisitos de admisibilidad. Link publicado en Teams.

Los hitos 3, 4, 5 y 12 son obligatorios porque sin ellos el trabajo no existe: definen el dominio, cargan los datos, resuelven la mitad del puntaje y constituyen la entrega. Los demás admiten hasta tres incumplimientos.

3.1 Los hitos se pueden hacer contra los datos semilla

Si el dominio propio del grupo todavía no está listo, el hito se cumple igual implementando el algoritmo sobre el grafo de ejemplo que trae el scaffold. Nadie queda bloqueado por el atraso de su grupo.

4. Calificación del TPO (10 puntos)

Los puntos se asignan por algoritmo implementado. Cada algoritmo debe estar accesible a través de un endpoint y documentado en docs/ENDPOINTS.md con ejemplos de entrada y salida.

Algoritmo	Hito	Puntos
Algoritmos sobre grafos: BFS y DFS	4	2
Dijkstra, Prim y Kruskal	5	3
Algoritmos Greedy	3	1
Divide y vencerás: QuickSort y MergeSort	2	1
Programación dinámica: mochila 0/1 o Floyd-Warshall	6 y 7	1
Backtracking	8	1
Ramificación y poda (Branch & Bound)	11	1
TOTAL		10

Los tres puntos de Dijkstra, Prim y Kruskal se reparten en 1 punto cada uno. Los dos puntos de BFS y DFS, en 1 punto cada uno.

4.1 Qué significa «implementado»

Nivel	Condiciones	Puntaje
Completo	Corre sin error · Da resultados correctos sobre los casos de verificación · Implementación propia, sin librerías · La estructura de datos elegida sostiene la complejidad declarada · Tiene al menos un test · Está documentado con su complejidad justificada	100 %
Parcial	Corre y da resultados correctos, pero falta el test, o la complejidad declarada no es la real, o la estructura elegida degrada el orden (por ejemplo, Dijkstra con búsqueda lineal del mínimo presentado como $O((V+E) \cdot \log V)$)	50 %
No cuenta	No corre · Da resultados incorrectos · Usa una librería que resuelve el algoritmo	0 %

4.2 Requisitos de admisibilidad

Si la entrega final no cumple estos seis puntos, se devuelve sin corregir y el grupo pasa a la instancia de regularización:

- Cumplió al menos 9 de los 12 hitos, incluidos los cuatro obligatorios.
- El proyecto compila y ./mvnw test pasa sin errores.

- No hay ninguna credencial en el repositorio: ni en el código, ni en `application.properties`, ni en el historial de commits.
- El README declara el dominio elegido, qué representa un nodo, qué representa una arista y en qué unidad está expresado el costo.
- Cada algoritmo declarado tiene su ficha en `docs/ENDPOINTS.md`.
- El historial de commits muestra participación de todos los integrantes.

4.3 Penalizaciones

Situación	Descuento
Lógica algorítmica dentro de un controller en lugar del paquete <code>algorithm</code>	-0,5
Un algoritmo que consulta la base de datos dentro de su bucle principal	-0,5 por algoritmo
Endpoint que devuelve un stack trace en lugar de un error manejado	-0,5
Hito entregado fuera de término	-0,25 por hito

5. Habilitación al Segundo Parcial

Regla

Para rendir el Segundo Parcial hay que tener el TPO aprobado con 4 o más puntos. La habilitación se comunica el 04/11 (clase 14), una semana antes del parcial.

5.1 Si un grupo no habilita

Nadie queda sin instancia de evaluación. El camino de recuperación es:

1. El 04/11 se comunica el estado de cada grupo y qué le falta concretamente.
2. El grupo tiene hasta el 18/11 (clase 16) para regularizar: completar los hitos obligatorios faltantes y corregir lo que impide la admisibilidad.
3. Regularizado el TPO, sus integrantes rinden el Segundo Parcial el 25/11, en la fecha del recuperatorio.
4. Si al 18/11 el TPO sigue sin aprobarse, el Segundo Parcial se considera desaprobado y se aplica el régimen de recuperatorio de la asignatura.

5.2 Un integrante puede habilitar aunque su grupo no

Si un grupo se desarma o un integrante deja de participar, quien sí cumplió sus hitos puede solicitar evaluación individual del trabajo realizado. El historial de commits es la evidencia. Hay que plantearlo antes del 21/10 (clase 12), no después.

6. Formato de la entrega

- Grupos de 3 o 4 integrantes, conformados en el hito 3 (19/08). No se aceptan grupos de 1, 2 ni de 5 o más.
- Repositorio en GitHub, con acceso de lectura para el docente si es privado.
- Un tag llamado `entrega-final` apuntando al commit que se corrige. Lo posterior a ese tag no se evalúa.
- Cada hito se entrega por Teams: link al commit correspondiente, antes de la clase siguiente.
- El README debe listar los integrantes con nombre completo y usuario de GitHub.

7. Uso de herramientas de inteligencia artificial

El uso de asistentes de IA está permitido y no requiere autorización previa. La materia los reconoce como una herramienta legítima de trabajo profesional. Hay tres condiciones.

1. Declararlo. El README debe incluir una sección «Uso de IA» indicando qué herramientas se usaron y para qué. No declararlo es una falta de honestidad académica.
2. El Segundo Parcial es la verificación. Se codifica en vivo, sin asistentes y sin acceso al propio repositorio. Quien no pueda escribir los algoritmos que entregó, no aprueba.
3. La comprensión no se delega. Generar el trabajo completo con una herramienta y presentarlo sin entenderlo se encuadra en el régimen disciplinario de deshonestidad académica.

El criterio en una frase

Usar IA para aprender más rápido está bien. Usarla para no aprender es lo que está en juego, y el Segundo Parcial existe justamente para distinguir una cosa de la otra.

Los actos de deshonestidad académica o cualquier situación de indisciplina serán sancionados según el régimen disciplinario correspondiente.